

Axios Interceptors

Two small functions decide everything: whether a failed request deserves a second try, and how to make sure two failures at once don't trigger two refreshes. Get this picture and the rest of the interceptor follows.

shouldAttemptRefresh

The skip-list check

Decides whether a 401 is even worth retrying.

- Returns **false** immediately if the request has no URL to check
- Skips `/auth/login`, `/auth/signup`, `/auth/refresh`
- A 401 on those means "wrong password" or "dead session" — not "expired token"
- Stops the refresh call's own failure from re-triggering itself

refreshAccessToken

The dedup guard

Makes sure simultaneous 401s only refresh once.

- Holds the in-flight request in one **shared, module-level** promise
- Calls plain **axios**, never the **api** instance — avoids recursion
- Every caller in the same window gets back the **same** promise
- Resets to **null** once the attempt settles, success or failure

THE MODULE

shouldAttemptRefresh(config)

runs first — a yes/no gate

refreshPromise (shared)

runs second — a dedup lock

Response interceptor

combines both, then retries

Both functions run **before** any retry happens — one decides **if**, the other makes sure it happens **only once**. Neither one calls axios again by itself; the interceptor combines them next.

Three conditions gate every retry

Before the interceptor touches the network, it checks three things. If **any one** is true, it gives up immediately — the original error passes straight through, exactly as if this file didn't exist.

1

Not a 401

500, a network error, 404 —
refreshing a token won't fix a server
crash

2

Already retried

originalRequest._retry is set
— tried once, still failed

3

On the skip-list

login / signup / refresh itself — a 401
there means something else

```
if (!isUnauthorized || alreadyRetried || !shouldAttemptRefresh(originalRequest)) {
  return Promise.reject(error); // re-throw, unchanged
}
```

Otherwise: refresh once, then retry

the happy path

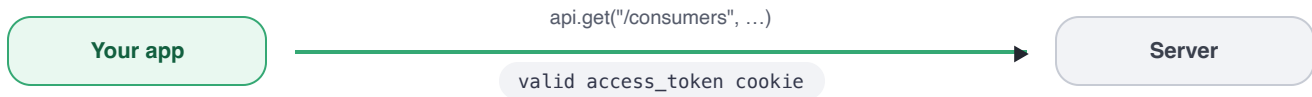
```
try {
  originalRequest._retry = true; // never retry the SAME request twice
  await refreshAccessToken(); // throws if the refresh token is also dead
  return api(originalRequest); // replay with the new cookie
} catch (refreshError) {
  window.location.href = "/login"; // hard redirect — no useNavigate() here
  return Promise.reject(refreshError);
}
```

- `_retry = true` marks this exact request so a repeat 401 hits gate 2 instead of looping.
- `api(originalRequest)` replays the same URL, method and body — the caller sees an ordinary success.

The full story, start to finish

Every hook in this app calls through the same instance, so this sequence runs identically no matter which request happens to hit an expired token.

1 A normal request



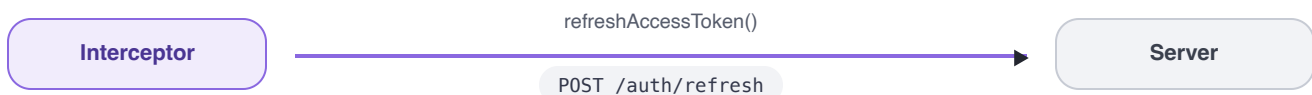
The common case — happens hundreds of times without this file ever doing anything visible.

2 Access token expires — 401



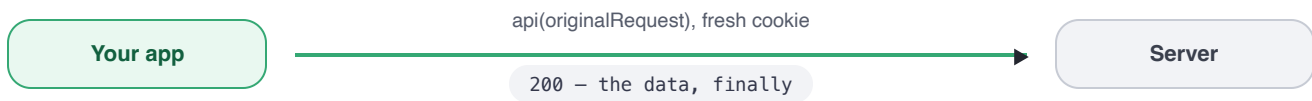
All three gate conditions on page 2 are checked here — all clear, so it proceeds.

3 Refresh, deduplicated



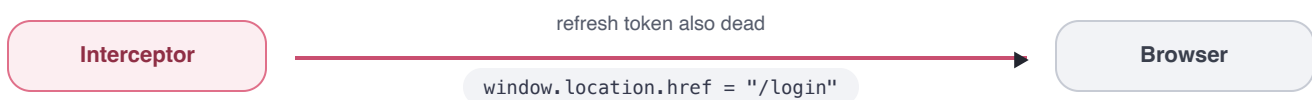
If 3 requests 401 at the same instant, all 3 share this one call instead of firing 3.

4 Silent retry — caller none the wiser



The component that made the original call never sees a failure — no error state, no re-render for it.

5 If refresh itself fails



A hard reload, not React Router — this code runs outside any component.